

16

Recipe assistant with dietary constraints

Build a retrieval-augmented generation system on a supplied corpus, under a constraint that defeats the naive pipeline.

Issued **18 August 2026**

Due **21 August 2026, 11:59 PM IST**

Effort **5–7 hours across 3 days**

fastembed

bge-small-en-v1.5

numpy cosine

gpt-4o-mini

no frameworks

Project 16 — Recipe assistant with dietary constraints

Your corpus

60 recipes with ingredients, cook time, veg/non-veg tags

Download the zip from the link in your email and unpack it as `data/` inside your repo. It contains:

```
recipes.md – 60 recipes with type, cook time, cuisine, ingredients
SOURCE.md
```

The question your system must answer

REPRESENTATIVE QUERY

Something vegetarian I can cook in 20 minutes with what's in my fridge.

Your constraint

This is what makes your project different from everyone else's. It is worth 25 of the 100 marks on its own, and it is the part that cannot be satisfied by running the workshop notebook unchanged.

REQUIRED

Hard metadata filter before similarity; similarity only ranks survivors. Constraints are buried in natural language, so extract them from the query first - rules or LLM, justify your choice.

PLANTED DIFFICULTY

Your corpus contains a deliberate trap — something built into these documents that defeats a straightforward implementation. Finding it is most of the assignment. If your system answers everything correctly on the first attempt, you have not found it yet. Name it in one line in your README.

What you are building

The six-block pipeline from the workshop — **Load** → **Chunk** → **Embed** → **Cosine Similarity** → **Retrieve** → **Ask** — pointed at your corpus, with your constraint applied.

You are not being asked to learn a new architecture. You are being asked to find where the workshop code breaks on real documents and fix it.

Stack — fixed, do not substitute

Layer	What to use
Embeddings	<code>fastembed</code> with <code>BAAI/bge-small-en-v1.5</code>
Similarity	<code>numpy</code> cosine similarity, hand-written
Chat	<code>aicredits.in/v1</code> , model <code>gpt-4o-mini</code>
Everything else	Standard library, <code>numpy</code> , <code>pandas</code>

NOT ALLOWED

LangChain, LlamaIndex, Haystack, Chroma, FAISS, Pinecone, or any framework that hides retrieval from you. If you cannot see the dot product, you cannot debug it. Frameworks come later in the cohort — this assignment is the layer underneath them.

The rule that outranks everything else

NON-NEGOTIABLE

If the answer is not in the retrieved chunks, your system says so. Never let the model fill the gap from its own knowledge. Your prompt must instruct refusal, and you must demonstrate it working on your two out-of-corpus questions.

Deliverables — four files

File	What it contains
<code>rag.ipynb</code>	Runs top to bottom on a fresh kernel. No hardcoded API keys.
<code>eval.md</code>	6 gold questions with expected answers, written before you tune anything. 2 of the 6 must be questions your system should refuse — answers genuinely not in your corpus. Report how many of the 6 you got right.
<code>README.md</code>	Under 300 words: your chunking decision and why, your parameter table, your score, and one line naming the planted difficulty you found.
<code>FAILURES.md</code>	150 words on your worst retrieval failure, what you tried, and whether you fixed it or worked around it.

READ THIS TWICE

An honest unfixed failure scores higher than a fixed one you cannot explain. Do not write `FAILURES.md` as a success story. It is the file that tells us whether you understood what happened.

Do not commit your API key. A key in git history means automatic resubmission.

How you are marked

Criterion	Marks
Retrieval works on the gold set	30
Your constraint implemented, not skipped	25
Eval set written before tuning, plus honest failure analysis	20
Code quality, README, reproducibility	15
Refusal behaviour	10

Marks lost automatically: hallucinated answers presented as grounded; an eval set clearly written after tuning; a chunk size chosen with no stated reason.

Three days — suggested shape

Day	Time	What to do
Day 1	~2 h	Load and read the corpus. Write <code>eval.md</code> — all six questions, before any tuning. Get the workshop pipeline running unchanged. Note what it gets wrong.
Day 2	2–3 h	Implement your constraint. Re-run the six questions. This is where the real work is.
Day 3	1–2 h	Refusal behaviour, README, FAILURES, clean the notebook, submit.

IF YOU GET STUCK

Post in the cohort group at the end of Day 1. Do not spend Day 2 stuck in silence — if you are stuck, three others are stuck on the same thing. Debugging questions are welcome. “Please send code” is not.

Submitting

1. Push your work to a GitHub repository.
2. Make it public, or add `admin@skillcred.in` as a collaborator.
3. Open the submission link in your assignment email and paste the repo URL.
4. You may update your submission any number of times before the deadline. The most recent one is marked.

Late submissions are accepted and recorded as late rather than refused. Submitting something incomplete on time is better than submitting nothing.

Integrity

Every participant has a different corpus and a different constraint. Discuss approach freely. Do not share code. Two submissions that resemble each other are not a coincidence and will be treated accordingly.

Questions: post in the cohort group. — Ganesan M, Co-Founder & CTO, SkillCred